

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ НИЖЕГОРОДСКОЙ ОБЛАСТИ
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
«НИЖЕГОРОДСКИЙ РАДИОТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

ОТЧЕТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ

ПМ.04 Сопровождение и обслуживание программного обеспечения компьютерных систем

Обучающегося

Яшина Дмитрия Ивановича
(Фамилия Имя Отчество)

Специальность: 09.02.07 Информационные системы и программирование

Квалификация: Программист

Курс: 4

Группа: 4ИСиП-22-2

Руководитель практики от ГБПОУ «НРТК» _____
(Должность)

(Фамилия Имя Отчество)

Нижний Новгород
2026 г.

Содержание

ПМ.04 Сопровождение и обслуживание программного обеспечения компьютерных систем.....	1
Введение.....	3
1. Краткая история создания предприятия.....	4
2. Анализ предметной области.....	5
3. Проектирование информационной системы.....	6
3.1 Диаграмма вариантов использования.....	6
3.2 Каркасная диаграмма.....	12
3.3 Блок-схемы работы алгоритмов реализованных в приложении.....	15
3.4 Диаграмма последовательности.....	18
3.5 Диаграмма состояний.....	20
3.6 Диаграмма компонентов.....	21
3.7 Диаграмма размещения.....	23
4. Разработка информационной системы.....	24
4.1 Выбор языка программирования.....	24
4.2 Разработка окна авторизации/регистрации.....	24
4.3 Разработка алгоритма рекомендательной системы.....	29
4.3.1 Алгоритм персональных рекомендаций.....	29
4.3.2 Алгоритм популярных треков.....	30
4.3.3 Эндпоинты рекомендаций.....	30
4.3.4 Интеграция с фронтендом.....	31
4.4 Разработка серверной части.....	31
4.4.1 Разработка базы данных.....	32
4.4.2 Сервис аутентификации.....	33
4.4.3 Сервис работы с треками.....	34
4.4.4 Сервис покупок и генерация договоров.....	35
4.4.5 Рекомендательная система.....	36
4.4.6 Личный кабинет пользователя и автора.....	36
4.4.7 Административная панель.....	37
4.5 Разработка клиентской части.....	37
4.5.1 Главная страница и каталог.....	38
4.5.2 Личный кабинет и профиль автора.....	39
4.5.3 Взаимодействие с сервером и аутентификация.....	39
Заключение.....	41

Введение

Цель практики:

Формирование общих компетенций: Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности. Планировать и реализовывать собственное профессиональное и личностное развитие. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами. Осуществлять устную и письменную коммуникацию на государственном языке с учетом особенностей социального и культурного контекста. Проявлять гражданско-патриотическую позицию, демонстрировать осознанное поведение на основе традиционных общечеловеческих ценностей. Содействовать сохранению окружающей среды, ресурсосбережению, эффективно действовать в чрезвычайных ситуациях. Использовать средства физической культуры для сохранения и укрепления здоровья в процессе профессиональной деятельности и поддержания необходимого уровня физической подготовленности. Использовать информационные технологии в профессиональной деятельности. Пользоваться профессиональной документацией на государственном и иностранном языке. Планировать предпринимательскую деятельность в профессиональной сфере.

Формирование профессиональных компетенций: Осуществлять установку, настройку и обслуживание программного обеспечения компьютерных систем. Осуществлять измерения эксплуатационных характеристик программного обеспечения компьютерных систем. Выполнять работы по модификации отдельных компонент программного обеспечения в соответствии с потребностями заказчика. Обеспечивать защиту программного обеспечения компьютерных систем программными средствами.

1. Краткая история создания предприятия

1992-1993: Создание компании. 30 декабря 1992 года распоряжением Госкомимущества создано государственное предприятие «Ростелеком». 27 августа 1993 года подписано распоряжение о его преобразовании в акционерное общество, а 23 сентября компания была официально зарегистрирована как АООТ «Ростелеком». В состав компании вошли 20 госпредприятий, обеспечивавших телевидение, междугородную и международную связь.

1994: Создание «Связьинвеста». В ноябре правительство РФ утвердило создание холдинга «Связьинвест», которому были переданы 38% акций «Ростелекома», закрепленных в федеральной собственности.

2002: Создание МРК. Произошло образование семи межрегиональных компаний связи (МРК) путем объединения региональных телекоммуникационных операторов.

2006: Утрата монополии. Компания лишилась монополии на рынке услуг дальней связи, которая была у нее с момента основания.

2011-2013: Крупномасштабная реорганизация. В 2011 году завершился первый этап реформы, в ходе которого к «Ростелекому» были присоединены МРК. В 2013 году произошло присоединение холдинга «Связьинвест» и еще 20 дочерних компаний, в результате чего было образовано единое юридическое лицо.

2014-2019: Развитие мобильного направления. В 2014 году «Ростелеком» и оператор Tele2 создали совместное предприятие на базе мобильных активов. В ноябре 2019 года «Ростелеком» консолидировал 100% Tele2 Россия.

2018: Новая стратегия. Компания начала реализацию новой стратегии до 2022 года, целью которой была трансформация в цифрового партнера для населения, бизнеса и государства.

2021: Стратегия до 2025 года. Совет директоров утвердил корпоративную стратегию до 2025 года, которая среди прочего ставила цель удвоения чистой прибыли.

2025: Новая дивидендная политика. В апреле совет директоров утвердил новую дивидендную политику на 2024–2026 отчетные годы, которая распространяется на выплаты в 2025, 2026 и 2027 годах

2. Анализ предметной области

Современная музыкальная индустрия требует специализированных инструментов для монетизации творчества и взаимодействия с аудиторией. Музыкальный маркетплейс — это веб-платформа, объединяющая авторов и слушателей (покупателей), позволяющая первым продавать свои работы, а вторым — приобретать их и получать персонализированные рекомендации.

Основными участниками маркетплейса являются:

Клиенты (слушатели, покупатели) — ищут музыку, слушают треки, добавляют в избранное, покупают и получают контракты на использование произведений.

Авторы — загружают треки, управляют каталогом, получают вознаграждение.

Администраторы — модерируют контент, управляют пользователями и анализируют статистику.

Ключевые автоматизируемые процессы:

Регистрация и аутентификация с разделением ролей.

Загрузка и хранение музыкальных файлов и метаданных.

Воспроизведение треков с учётом прослушиваний.

Система избранного.

Покупка трека с выбором лицензии и генерацией договора.

Персональные рекомендации на основе предпочтений.

Сбор и визуализация статистики продаж и активности.

Традиционный способ распространения музыки (лейблы, файлообменники) имеет недостатки:

Отсутствие прозрачности для независимых авторов.

Слабая персонализация контента.

Юридические риски (нет автоматических договоров).

Фрагментация данных.

Разрабатываемая система «BeatMarket» решает эти проблемы за счёт клиент-серверной архитектуры, разграничения прав, аудиоплеера, системы рекомендаций и административной панели.

Таким образом, предметная область характеризуется высокой динамикой контента, необходимостью обработки мультимедиа, сложными взаимосвязями сущностей и потребностью в интеллектуальных алгоритмах рекомендаций. Успешная реализация требует надёжного хранения данных, эффективных алгоритмов и удобного интерфейса.

3. Проектирование информационной системы

Для проектирования данной информационной системы выбран язык объектно-ориентированного подхода UML - это язык графического описания, предназначенный для объектного моделирования бизнес-процессов, для моделирования в области программного обеспечения, для системного проектирования и отображения организационных структур.

3.1 Диаграмма вариантов использования

Диаграмма прецедентов (англ. use-case diagram) - диаграмма, описывающая, какой функционал разрабатываемой программной системы доступен каждой группе пользователей. Диаграммы вариантов использования UML являются основной формой требований к системе/программному обеспечению для новых разрабатываемых программ. Ключевой концепцией моделирования вариантов использования является то, что оно помогает разработчику проектировать систему с точки зрения конечного пользователя. В разрабатываемой системе выявлены основные актеры: учитель, ученик.

На рисунке 3.1 представлена диаграмма прецедентов системы для сайта по продаже музыки.

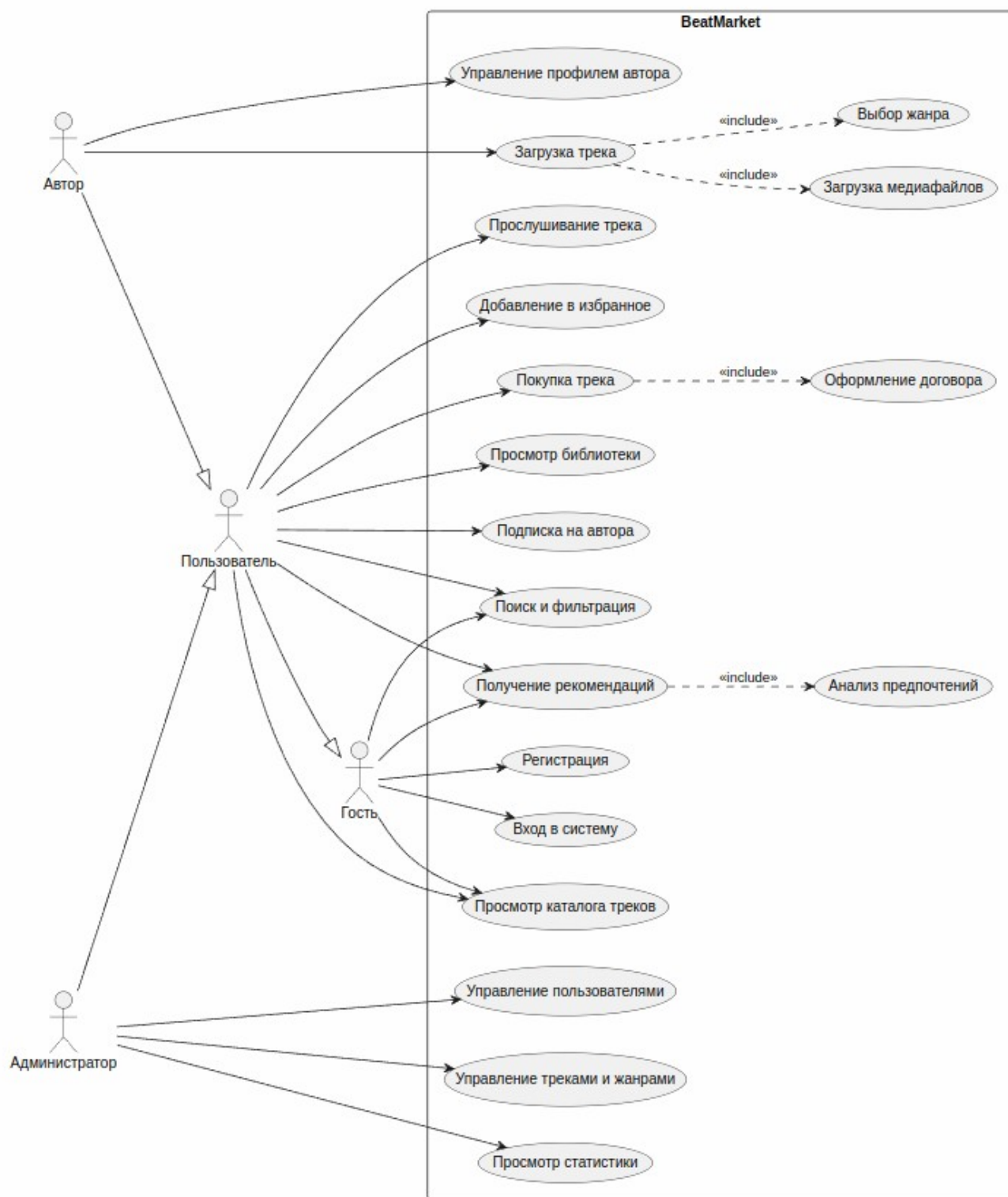


Рис 3.1 Диаграмма прецедентов

В таблицах 3.1 – 3.9 представлены описательные спецификации прецедентов, представленных на диаграмме.

Таблица 3.1

Описательная спецификация прецедента «Регистрация и авторизация»

Прецедент	Регистрация
Краткое описание	Процесс создания новой учётной записи для получения доступа к функционалу музыкального маркетплейса.
Субъекты	Гость
Предусловие	Приложение запущено, пользователь находится на странице регистрации.
Основной поток	Пользователь заполняет поля: ФИО, логин, пароль, адрес электронной почты. Система проверяет уникальность логина и корректность email. После успешной валидации создаётся новая запись в таблице <code>User</code> .
Постусловие	Учётная запись добавлена в базу данных. Пользователь перенаправляется на страницу входа.

Таблица 3.2

Описательная спецификация прецедента «Вход в систему»

Прецедент	Вход в систему
Краткое описание	Аутентификация пользователя для получения доступа к закрытым разделам платформы.
Субъекты	Гость
Предусловие	Приложение запущено, пользователь находится на странице входа.
Основной поток	Пользователь вводит логин и пароль. Система проверяет наличие пользователя в БД и корректность пароля. В случае успеха генерируется JWT-токен и сохраняется на клиенте.
Постусловие	Пользователь авторизован, получает доступ к личному кабинету и функционалу в соответствии с ролью.

Таблица 3.3

Описательная спецификация прецедента «Просмотр каталога треков»

Прецедент	Просмотр каталога треков
Краткое описание	Навигация по списку доступных музыкальных произведений, представленных на платформе.
Субъекты	Гость, Пользователь, Автор, Администратор

Предусловие	Пользователь находится на главной странице или в разделе каталога.
Основной поток	Система отправляет запрос к серверу, получает список треков с пагинацией и отображает их в виде карточек (обложка, название, автор, цена).
Постусловие	Пользователь видит актуальный список треков, может взаимодействовать с элементами карточек (слушать, купить, добавить в избранное).

Таблица 3.4

Описательная спецификация прецедента «Поиск и фильтрация»

Прецедент	Поиск и фильтрация
Краткое описание	Поиск треков по названию, фильтрация по жанру и автору для уточнения результатов.
Субъекты	Гость, Пользователь, Автор, Администратор
Предусловие	Пользователь находится в каталоге треков.
Основной поток	Пользователь вводит поисковый запрос в строку поиска или выбирает значения в фильтрах (жанр, автор). Система формирует соответствующий запрос к БД и обновляет отображаемый список треков.
Постусловие	Пользователь видит отфильтрованный / найденный список треков.

Таблица 3.5

Описательная спецификация прецедента «Прослушивание трека»

Прецедент	Прослушивание трека
Краткое описание	Воспроизведение аудиофайла музыкального произведения через встроенный плеер.
Субъекты	Пользователь, Автор, Администратор
Предусловие	Пользователь авторизован, выбран конкретный трек.
Основной поток	Пользователь нажимает кнопку «Слушать». Система отправляет GET-запрос на endpoint <code>/stream/{track_id}</code> , получает MP3-файл и начинает его воспроизведение. После начала прослушивания отправляется POST-запрос на увеличение счётчика <code>plays</code> и создание записи в таблице <code>Interaction</code> .

Постусловие	Трек воспроизведён, статистика прослушиваний обновлена на сервере.
-------------	--

Таблица 3.6

Описательная спецификация прецедента «Добавление в избранное»

Прецедент	Добавление в избранное
Краткое описание	Сохранение понравившегося трека в персональный список избранного.
Субъекты	Пользователь, Автор
Предусловие	Пользователь авторизован и находится на странице трека или в каталоге.
Основной поток	Пользователь нажимает на иконку сердца у трека. Система проверяет, не добавлен ли уже этот трек в избранное. Если нет — создаётся запись в таблице <code>Favorite</code> . Если трек уже в избранном — запись удаляется.
Постусловие	Статус трека в избранном изменён, данные синхронизированы с сервером.

Таблица 3.7

Описательная спецификация прецедента «Покупка трека»

Прецедент	Покупка трека
Краткое описание	Приобретение прав на использование музыкального произведения с автоматическим оформлением договора.
Субъекты	Пользователь, Автор
Предусловие	Пользователь авторизован, трек ранее не был куплен.
Основной поток	Пользователь нажимает кнопку «Купить», выбирает тип лицензии. Система проверяет наличие трека в библиотеке пользователя. Если трек не куплен, создаётся запись в таблице <code>Purchase</code> , генерируется уникальный номер договора и создаётся запись в <code>Contract</code> .
Постусловие	Трек добавлен в библиотеку пользователя, договор доступен для скачивания.

Таблица 3.8

Описательная спецификация прецедента «Просмотр библиотеки»

Прецедент	Просмотр библиотеки
Краткое описание	Отображение всех купленных пользователем треков в отдельном

	разделе.
Субъекты	Пользователь, Автор
Предусловие	Пользователь авторизован.
Основной поток	Пользователь переходит в раздел «Моя библиотека». Система формирует запрос к БД на получение всех покупок пользователя и отображает их в виде списка или сетки.
Постусловие	Пользователь видит свои приобретённые треки и может их прослушивать.

Таблица 3.9

Описательная спецификация прецедента «Загрузка трека»

Прецедент	Загрузка трека
Краткое описание	Публикация нового музыкального произведения автором.
Субъекты	Автор
Предусловие	Пользователь авторизован с ролью «Автор», находится в разделе загрузки.
Основной поток	Автор заполняет форму: название, жанр, цена, длительность, ВРМ (опционально), загружает МРЗ-файл и обложку. Система валидирует данные и сохраняет трек в БД, а файлы — в файловое хранилище.
Постусловие	Новый трек появляется в каталоге и доступен для прослушивания и покупки.

3.2 Каркасная диаграмма

Каркасная диаграмма (wireframe) представляет собой упрощённое визуальное представление структуры пользовательского интерфейса, на котором отображается расположение ключевых функциональных блоков, элементов навигации и управления без детальной проработки графического оформления. Каркасные диаграммы используются на этапе проектирования для согласования компоновки экранов, уточнения сценариев взаимодействия и демонстрации общей логики интерфейса до начала разработки.

В рамках дипломного проектирования с помощью инструмента PlantUML Salt разработаны каркасные диаграммы четырёх основных экранов информационной системы BeatMarket: страницы входа, главной страницы, страницы поиска и личного кабинета пользователя. Каждая диаграмма отражает функциональные требования, заложенные в техническом задании.

На диаграмме (рисунок 3.2.1) представлена форма аутентификации пользователя. Она включает поля ввода логина и пароля, чекбокс «Запомнить меня» и кнопку входа. В нижней части расположена ссылка для перехода на страницу регистрации. Данная компоновка соответствует требованиям разграничения доступа и обеспечивает простой и интуитивно понятный вход в систему.

BeatMarket - Вход

^ Login

^ Password

☒ Remember me

Нет аккаунта? [\[Зарегистрироваться\]](#)

Рис 3.2.1 – Каркасная диаграмма страницы входа

На рисунке 3.2.2 изображён макет главной страницы, выполняющей роль точки входа в каталог музыкальных произведений. В верхней части размещена панель поиска и фильтрации, а также иконка профиля. Основное пространство занимают три секции: «Популярные биты», «Подборки по жанрам» и «Новые релизы». Каждый блок содержит карточки треков с обложкой, названием, именем автора и кнопками прослушивания и покупки. Такая структура позволяет пользователю быстро ознакомиться с актуальным контентом и перейти к целевому действию.



Рис 3.2.2 – Каркасная диаграмма главной страницы

Диаграмма на рисунке 3.2.3 демонстрирует экран расширенного поиска и фильтрации. Интерфейс разделён на две колонки: левая содержит панель фильтров (жанры, диапазоны BPM и длительности, кнопка применения), а правая – область результатов с возможностью сортировки и переключения режима отображения (сетка/список). В нижней части правой колонки показаны карточки треков, соответствующие заданным критериям. Подобная компоновка реализует требование технического задания о поиске и фильтрации музыкальных произведений по различным параметрам.



Рис 3.2.3 – Каркасная диаграмма страницы поиска

На рисунке 3.2.4 представлен макет личного кабинета авторизованного пользователя (в режиме автора). Верхняя часть отображает аватар, имя, роль и базовую статистику (количество треков, подписчиков, покупок). Ниже расположены вкладки для навигации по разделам: «Мои данные», «Покупки», «Избранное», «Подписки», «Студия». Основное содержимое вкладки «Мои данные» включает персональную информацию и кнопку редактирования профиля, а также область для статистических графиков. Такая организация соответствует требованиям к личному кабинету клиента и панели автора, обеспечивая удобный доступ ко всем функциям управления аккаунтом и контентом.

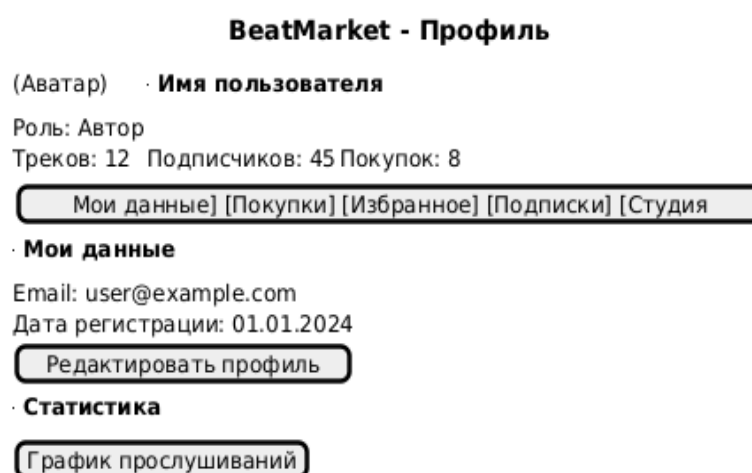


Рис 3.2.4 – Каркасная диаграмма личного кабинета

3.3 Блок-схемы работы алгоритмов реализованных в приложении

Блок-схема представляет собой графическое отображение логической последовательности действий, выполняемых программой для решения конкретной задачи. Она позволяет наглядно представить структуру алгоритма, ветвления и циклы, что особенно важно при проектировании сложных бизнес-процессов информационной системы. Блок-схемы служат инструментом для документирования ключевых функций приложения, упрощают понимание логики работы как для разработчиков, так и для проверяющих, а также облегчают последующую отладку и модернизацию кода.

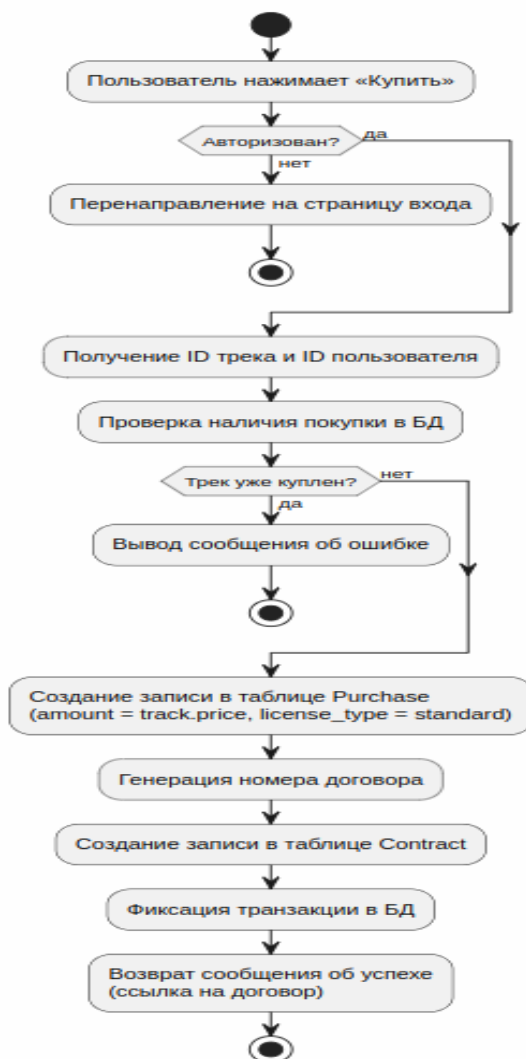


Рисунок 3.3.1 Блок-схема покупки трека

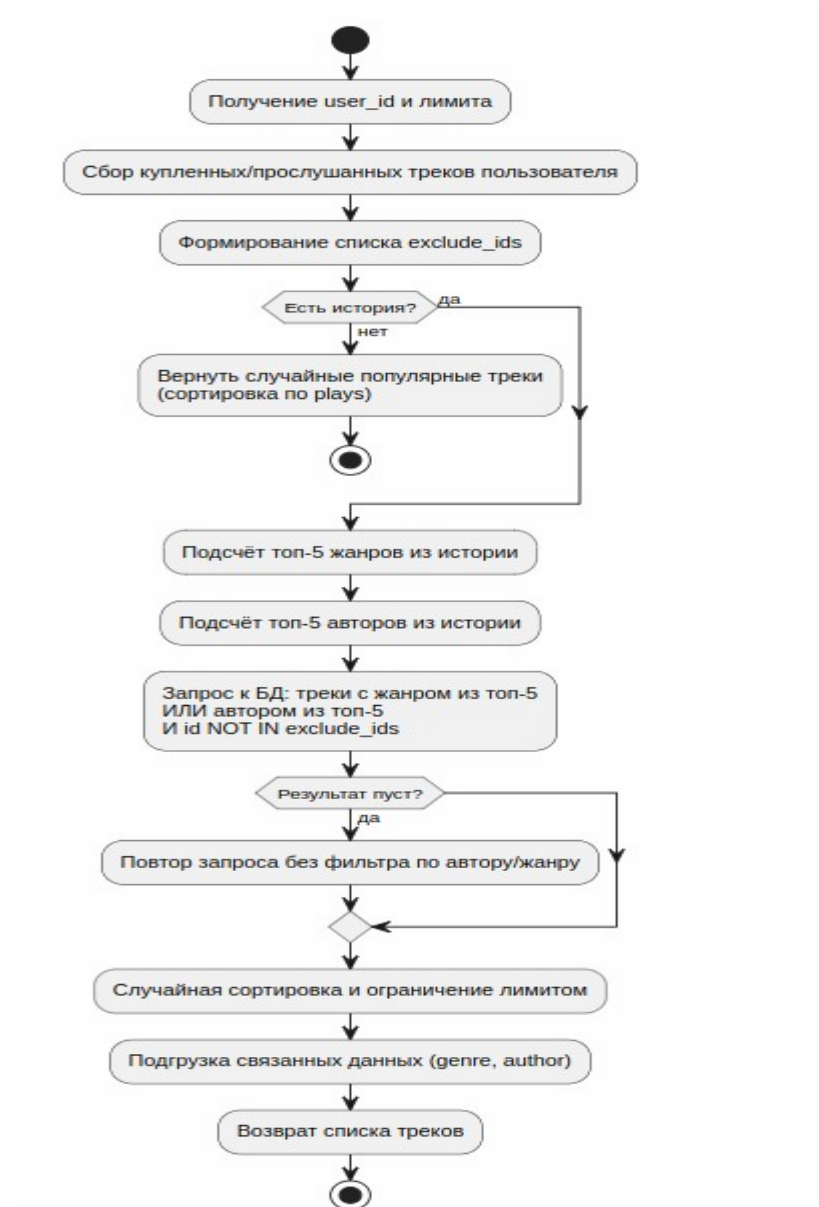


Рисунок 3.3.2 Блок-схема персональных рекомендаций

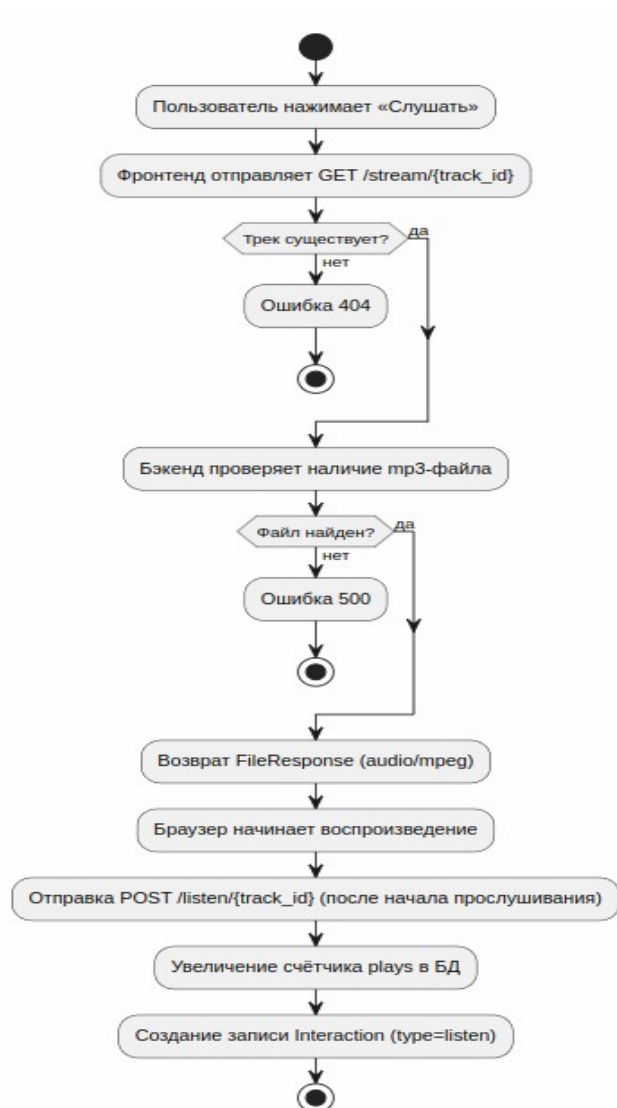


Рисунок 3.3.3 Блок-схема потокового прослушивания

3.4 Диаграмма последовательности

Диаграммы последовательности, обычно моделируют взаимодействия между объектами в едином сценарии использования. Они иллюстрируют, как различные части системы взаимодействуют друг с другом для выполнения функции, а также порядок, в котором происходит взаимодействие при выполнении конкретного случая использования.

Схема последовательности построена таким образом, что она представляет собой временную шкалу, которая начинается сверху и постепенно опускается, чтобы отметить последовательность взаимодействий. Каждый объект имеет колонку, а сообщения, которыми обмениваются между собой, представлены стрелками.

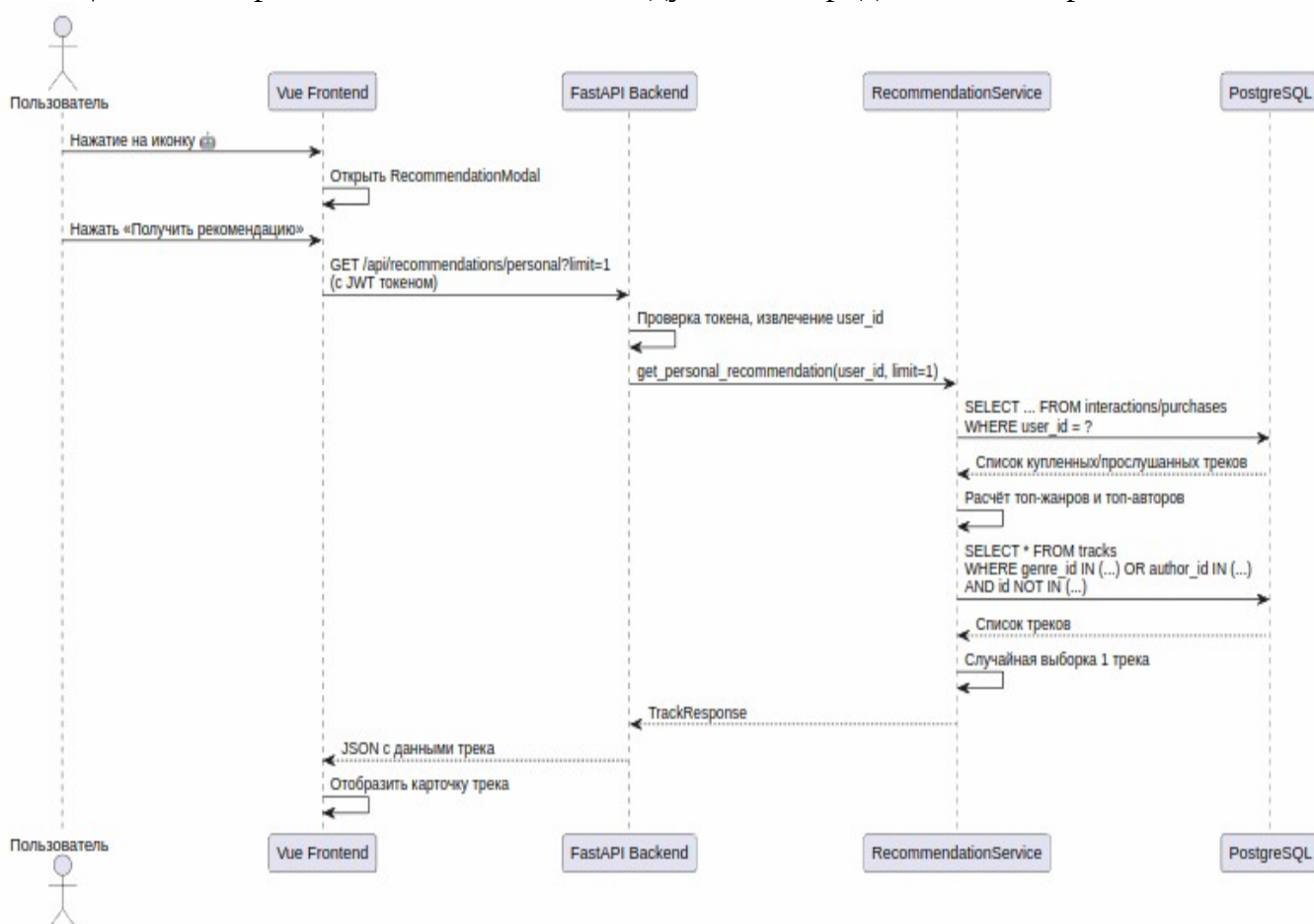


Рис 3.4.1 Диаграмма последовательности

Процесс получения персональной рекомендации состоит из следующих последовательных шагов:

- **Инициация:** Пользователь, находясь в любом разделе приложения, нажимает на иконку в верхней панели навигации. Фронтенд-приложение на Vue.js перехватывает это событие и отображает модальное окно RecommendationModal с помощью механизма Teleport, не перезагружая страницу.
- **Запрос рекомендации:** Пользователь нажимает кнопку «Получить рекомендацию» внутри модального окна. Компонент формирует асинхронный

HTTP GET-запрос к эндпоинту `/api/recommendations/personal` с параметром `limit=1`. В заголовок `Authorization` автоматически добавляется JWT-токен, сохранённый в локальном хранилище браузера после авторизации.

- Аутентификация на сервере: Запрос поступает на FastAPI-сервер и перехватывается зависимостью `get_current_user`. Происходит валидация JWT-токена: проверяется подпись, срок действия и целостность. В случае успеха из полезной нагрузки токена извлекается идентификатор пользователя (`user_id`), который становится доступным для последующих слоёв приложения.
- Передача в сервисный слой: Роутер `RecommendationRouter` передаёт управление и извлечённый `user_id` в метод `get_personal_recommendation` сервиса `RecommendationService`. Этот слой инкапсулирует всю бизнес-логику, связанную с формированием рекомендаций.
- Сбор истории пользователя: Сервис через соответствующие репозитории (`PurchaseRepository`, `InteractionRepository`) выполняет два SQL-запроса к базе данных PostgreSQL. Первый запрос извлекает все треки, купленные пользователем (таблица `purchases`). Второй запрос извлекает все треки, прослушанные пользователем (таблица `interactions` с типом `listen`). База данных возвращает два набора записей, содержащих идентификаторы треков, их жанры и идентификаторы авторов.
- Анализ предпочтений: Получив историю, `RecommendationService` производит агрегацию данных: подсчитывает частоту встречаемости каждого жанра и каждого автора в истории пользователя. Из полученных распределений выбираются топ-5 жанров и топ-5 авторов, которые представляют собой ядро музыкальных предпочтений пользователя. Параллельно формируется список `exclude_ids`, включающий идентификаторы всех треков, которые пользователь уже купил или слушал — их повторная рекомендация нежелательна.
- Формирование выборки кандидатов: Если история пользователя не пуста, сервис формирует новый SQL-запрос к таблице `tracks`. Запрос включает условия: `genre_id` должен входить в список топ-5 жанров ИЛИ `author_id` должен входить в список топ-5 авторов. Дополнительно накладывается условие `id NOT IN (exclude_ids)`, исключающее уже известные треки. Для обеспечения разнообразия к результату применяется случайная сортировка (`ORDER BY random()`) и ограничение выборки параметром `limit`.
- Обработка крайних случаев: Если пользователь новый и его история пуста, сервис переключается на стратегию «популярные треки» и выполняет запрос на выборку треков с наибольшим количеством прослушиваний (`plays`). Если запрос с фильтрацией по жанрам и авторам не вернул ни одной записи, выполняется повторный запрос с ослабленными условиями — фильтр по жанрам и авторам снимается, чтобы гарантированно предоставить пользователю хоть какой-то трек.
- Подгрузка связанных данных: Получив итоговый список идентификаторов треков, сервис выполняет финальный запрос к базе данных для извлечения полных объектов `Track`. При этом с помощью механизма `selectinload` библиотеки `SQLAlchemy` в одном запросе подгружаются связанные сущности:

объект Genre (жанр) и объект Author (автор) для каждого трека. Это исключает проблему «N+1 запросов» и оптимизирует взаимодействие с БД.

- Сериализация и возврат ответа: Сервис возвращает объекты треков в роутер, где они преобразуются в Pydantic-схему TrackResponse. Сервер отправляет клиенту HTTP-ответ со статусом 200 (OK), содержащий JSON-массив с данными одного трека (название, автор, обложка, цена, длительность и т.д.).
- Завершение на клиенте: Фронтенд-приложение получает ответ, парсит JSON и обновляет состояние компонента RecommendationModal. В модальном окне отображается карточка трека с обложкой, названием, именем автора и ценой. Пользователь может взаимодействовать с предложенным треком: прослушать его через встроенный плеер, перейти к покупке или запросить другой вариант, нажав кнопку «Другой трек», что инициирует повторение описанного процесса с обновлённым списком исключённых идентификаторов.

3.5 Диаграмма состояний

Диаграмма состояний (Statechart diagram) показывает, как объект переходит из одного состояния в другое. Диаграммы состояний служат для моделирования динамических аспектов системы. Данная диаграмма полезна при моделировании жизненного цикла объекта.

От других диаграмм диаграмма состояний отличается тем, что описывает процесс изменения состояний только одного экземпляра определенного класса - одного объекта, причем объекта реактивного, то есть объекта, поведение которого характеризуется его реакцией на внешние события.



Рис 3.5.1 Диаграмма состояний — создание покупки

3.6 Диаграмма компонентов

Диаграмма компонентов (англ. Component diagram) — элемент языка моделирования UML, статическая структурная диаграмма, которая показывает разбиение программной системы на структурные компоненты и связи (зависимости) между компонентами.

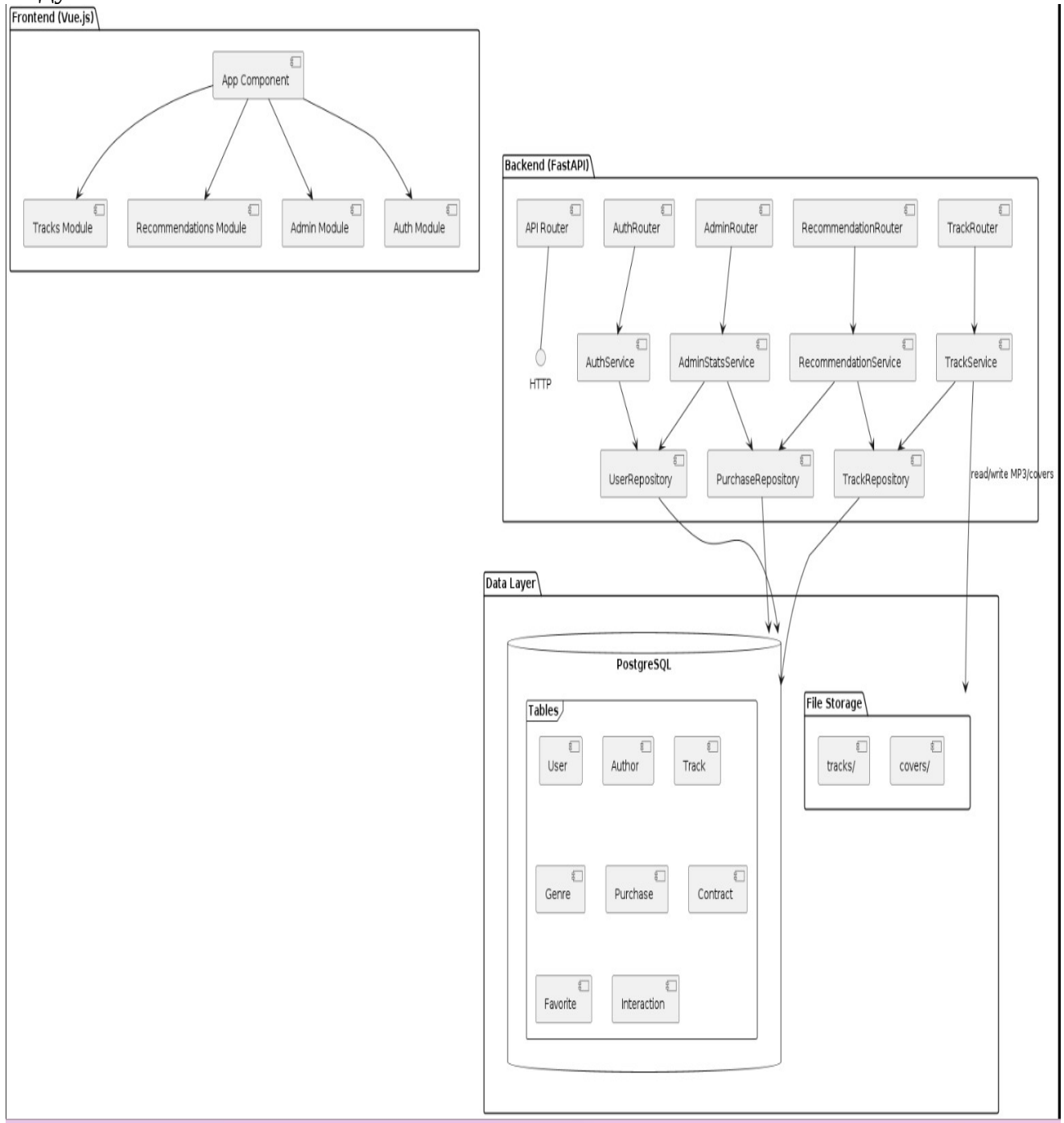


Рис 3.6.1 Диаграмма компонентов

На диаграмме выделены три глобальных подсистемы (уровня архитектуры) и внутренние программные модули, из которых они состоят:

1. Подсистема «Клиентское веб-приложение (Vue.js)»

Представляет собой одностраничное приложение (SPA), исполняемое в браузере конечного пользователя. Внутренняя архитектура клиента разделена на

следующие саб-компоненты:

App Component — корневой компонент, управляющий жизненным циклом приложения, маршрутизацией и глобальным состоянием.

Auth Module — модуль аутентификации, отвечающий за формы входа и регистрации, а также за хранение и обновление JWT-токенов в локальном хранилище браузера.

Tracks Module — модуль каталога треков, обеспечивающий отображение списка музыкальных произведений, поиск, фильтрацию и взаимодействие с встроенным аудиоплеером.

Recommendations Module — модуль персональных рекомендаций, реализующий модальное окно с предложением треков и взаимодействие с соответствующими эндпоинтами API.

Admin Module — модуль административной панели, предоставляющий интерфейс для управления пользователями, треками, жанрами и просмотра статистики.

2. Подсистема «Серверное приложение (FastAPI)»

Микросервис на базе асинхронного фреймворка FastAPI, предоставляющий (provided interface) стандартизированный REST API для клиентского приложения. Внутренняя структура включает:

API Router — корневой маршрутизатор, объединяющий все эндпоинты приложения и обеспечивающий единую точку входа для HTTP-запросов.

AuthRouter — маршрутизатор аутентификации, обрабатывающий запросы на регистрацию, вход и обновление токенов.

TrackRouter — маршрутизатор треков, отвечающий за CRUD-операции с музыкальными произведениями, а также за потоковую передачу MP3-файлов.

RecommendationRouter — маршрутизатор рекомендаций, предоставляющий эндпоинты для получения популярных и персональных рекомендаций.

AdminRouter — маршрутизатор административной панели, обеспечивающий доступ к управлению пользователями и статистическим отчётам.

Сервисный слой (AuthService, TrackService, RecommendationService, AdminStatsService) — ядро бизнес-логики приложения. Инкапсулирует правила обработки данных, включая проверку прав доступа, расчёт статистических метрик, алгоритмы рекомендаций и генерацию договоров.

Слой репозитория (UserRepository, TrackRepository, PurchaseRepository) — абстрагирует сервисный слой от конкретного диалекта SQL, преобразуя вызовы методов Python в ORM-запросы SQLAlchemy. Требуется (required interface) наличия подключения к СУБД PostgreSQL.

3. Подсистема «Слой данных»

Включает два независимых компонента хранения информации:

СУБД PostgreSQL — реляционная база данных, предоставляющая интерфейс взаимодействия по протоколу TCP/IP. Отвечает за персистентное хранение структурированных данных (пользователи, треки, покупки, жанры и т.д.), контроль целостности внешних ключей и обеспечение ACID-транзакций. Внутри базы данных выделены таблицы: User, Author, Track, Genre, Purchase, Contract,

Favorite, Interaction.

Файловое хранилище — директория на сервере, предназначенная для хранения бинарных файлов (MP3-треки в папке tracks/ и изображения обложек в папке covers/). Доступ к файлам осуществляется напрямую из сервисного слоя (TrackService) для операций чтения и записи, а также через статическую раздачу FastAPI для клиентского приложения.

3.7 Диаграмма размещения

Диаграмма размещения (deployment diagram) — это диаграмма в языке UML, которая показывает конфигурацию обрабатывающих узлов и артефактов, размещенных в этих узлах. Она отражает физические взаимосвязи между программными и аппаратными компонентами системы и помогает визуализировать, как программное обеспечение будет работать на разных устройствах.

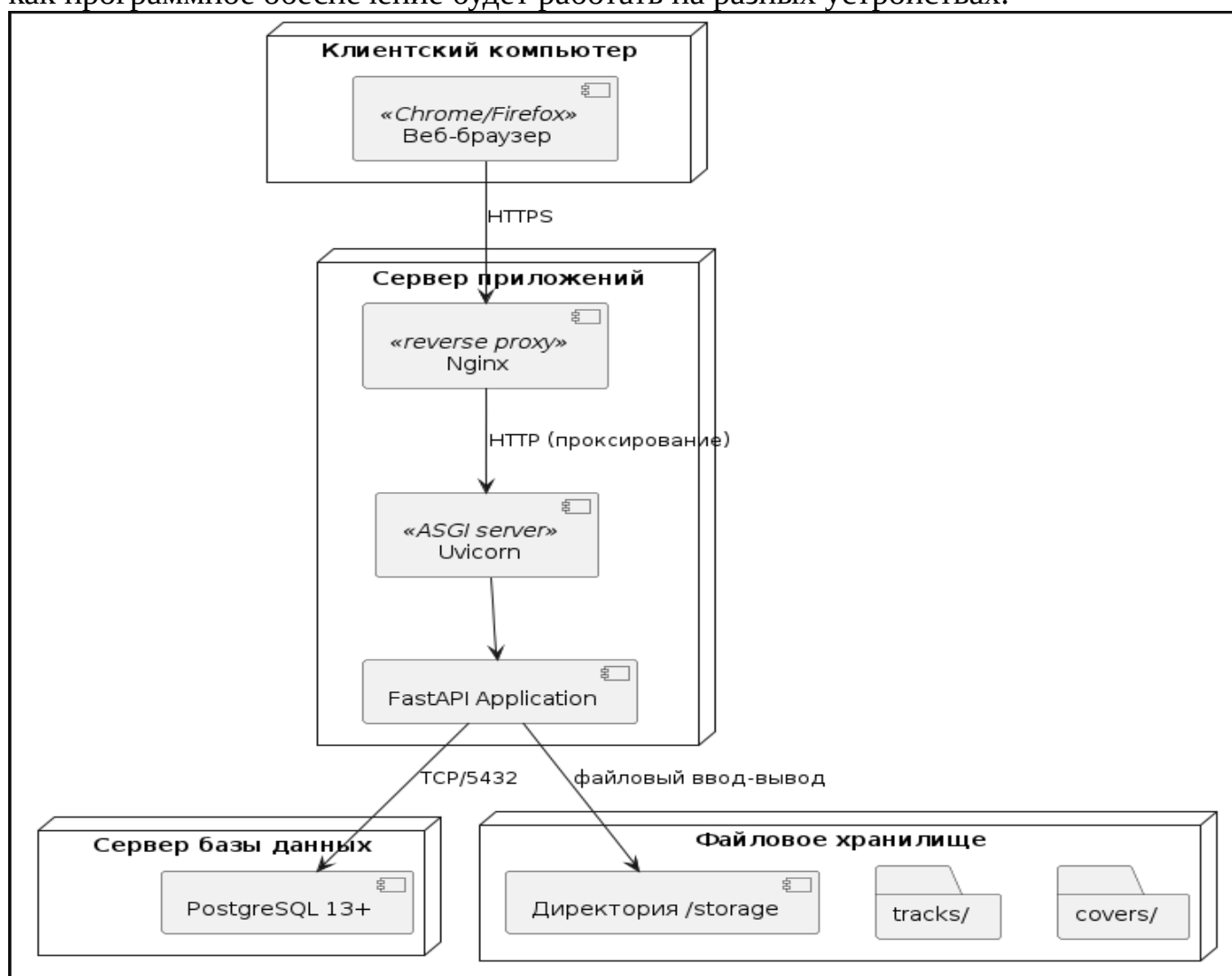


Рис 3.7.1 диаграмма размещения

4. Разработка информационной системы

4.1 Выбор языка программирования

Python — это высокоуровневый объектно-ориентированный язык программирования, разработанный для создания надежных и легко читаемых приложений. Он является одним из самых популярных языков программирования в мире благодаря своей стабильности, кроссплатформенности и широкой применимости.

Разработка на языке Python позволяет создавать как простые скрипты, так и сложные корпоративные клиент-серверные системы. Один из ключевых плюсов Python — богатая экосистема библиотек, что делает его идеальным выбором для проектов, где важна скорость разработки и масштабируемость.

Python подходит для решения задач различной сложности:

- Создание современных асинхронных веб-серверов и API (FastAPI, Django, Flask).
- Построение микросервисных архитектур.
- Интеграция с базами данных (SQLAlchemy ORM).

Таким образом, серверную часть проекта лучше всего разрабатывать на языке программирования Python, поскольку он позволяет:

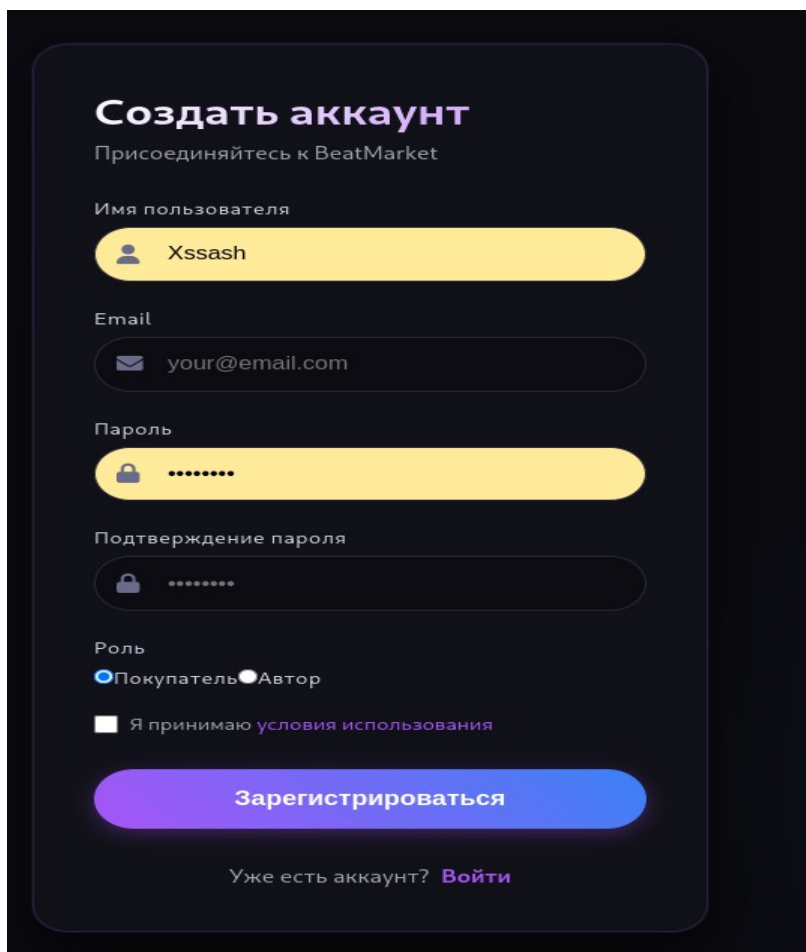
- Реализовать высоконагружаемую асинхронную серверную часть с использованием FastAPI.
- Интегрировать базу данных PostgreSQL через ORM-библиотеки (SQLAlchemy).
- Обеспечить безопасность персональных данных (хэширование паролей, JWT-токены).

В соответствии с поставленной целью, необходимо рассмотреть:

- Использование фреймворка FastAPI для реализации REST API и взаимодействия с базой данных.
- Хранение и обработку данных с использованием СУБД PostgreSQL.

4.2 Разработка окна авторизации/регистрации

Окно авторизации является точкой входа на сайт. Оно обеспечивает идентификацию пользователя по паре «email – пароль», а также предоставляет возможность регистрации нового пользователя с проверкой корректности вводимых данных. Такой подход гарантирует безопасность доступа и целостность учётных записей.



Создать аккаунт
Присоединяйтесь к BeatMarket

Имя пользователя

Email

Пароль

Подтверждение пароля

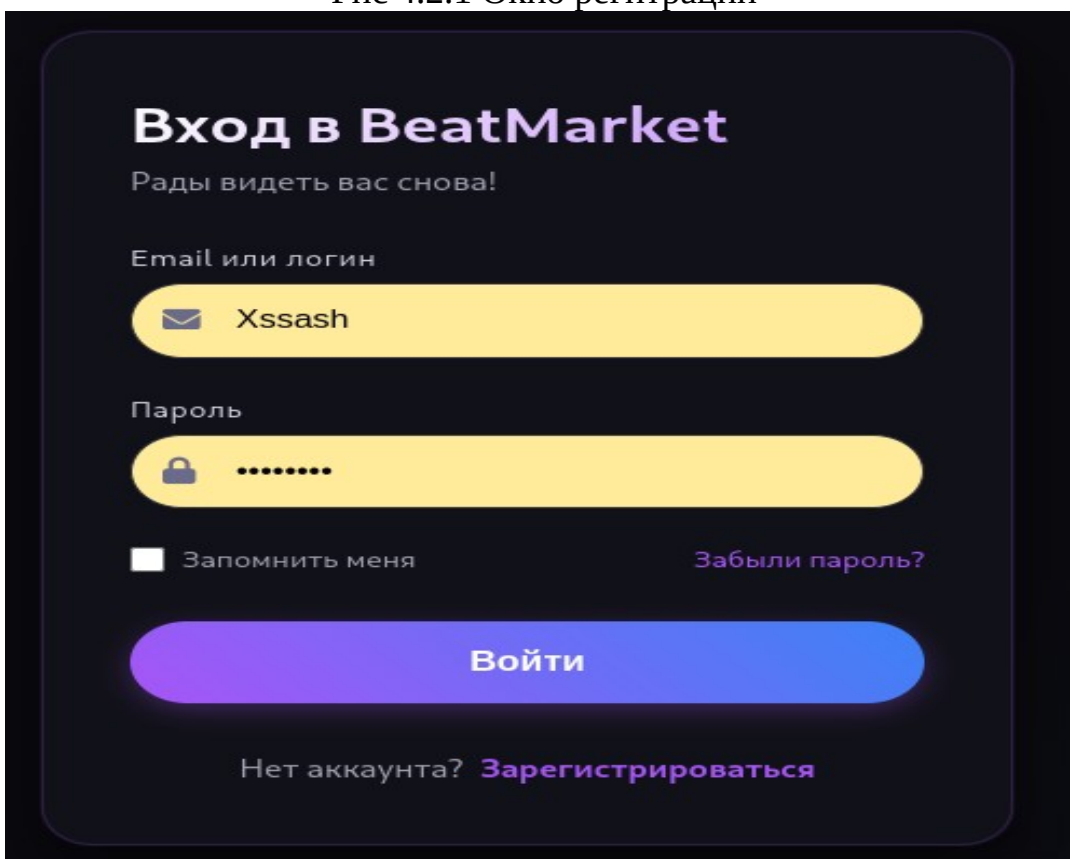
Роль
☒ Покупатель ☐ Автор

☐ Я принимаю условия использования

Зарегистрироваться

Уже есть аккаунт? [Войти](#)

Рис 4.2.1 Окно регистрации



Вход в BeatMarket
Рады видеть вас снова!

Email или логин

Пароль

☐ Запомнить меня [Забыли пароль?](#)

Войти

Нет аккаунта? [Зарегистрироваться](#)

Рис 4.2.2 Окно авторизации

1. Регистрация пользователя

При заполнении формы регистрации (**Register.vue**) пользователь вводит имя, email, пароль и выбирает роль. Клиентская валидация проверяет совпадение паролей и обязательное принятие условий.

```
<!-- Register.vue: обработчик отправки формы -->
<script setup>
const handleRegister = async () => {
  if (password.value !== confirmPassword.value) {
    alert('Пароли не совпадают');
    return;
  }
  if (!agree.value) {
    alert('Примите условия использования');
    return;
  }
  try {
    await authStore.register({
      full_name: username.value,
      login: username.value,
      email: email.value,
      password: password.value,
      role: role.value
    });
    router.push('/');
  } catch (error) {
    alert('Ошибка регистрации: ' + (error.response?.data?.detail ||
    'Неизвестная ошибка'));
  }
};
</script>
```

После успешной валидации вызывается метод **authStore.register**, отправляющий POST-запрос на **/auth/register**. Серверный эндпоинт обрабатывает запрос, создаёт пользователя и сразу устанавливает токены в куки.

```
# auth.py (FastAPI router)
@router.post("/register", response_model=UserResponse,
status_code=status.HTTP_201_CREATED)
async def register(
    user_data: UserCreate,
    response: Response,
    auth_service: AuthService = Depends(get_auth_service)
):
    """Регистрация нового пользователя и автоматический вход"""
    user = await auth_service.register(user_data)
    tokens = await auth_service.create_tokens(user.id)
    set_auth_cookies(response, tokens["access_token"],
tokens["refresh_token"])
    return user
```

Функция **set_auth_cookies** помещает токены в httpOnly куки с заданными параметрами безопасности.

```
def set_auth_cookies(response: Response, access_token: str, refresh_token:
```

```

str):
    response.set_cookie(
        key="access_token",
        value=access_token,
        httponly=True,
        secure=COOKIE_SECURE,
        samesite=COOKIE_SAMESITE,
        max_age=ACCESS_TOKEN_EXPIRE,
    )
    response.set_cookie(
        key="refresh_token",
        value=refresh_token,
        httponly=True,
        secure=COOKIE_SECURE,
        samesite=COOKIE_SAMESITE,
        max_age=REFRESH_TOKEN_EXPIRE,
    )

```

2. Вход в систему

Форма входа (**Login.vue**) содержит поля логин/email и пароль. При нажатии кнопки «Войти» вызывается **handleLogin**, который через **authStore.login** отправляет POST-запрос на **/auth/login**.

```

<!-- Login.vue: обработчик входа -->
<script setup>
const handleLogin = async () => {
    if (!login.value || !password.value) {
        errorMessage.value = 'Заполните все поля';
        return;
    }
    loading.value = true;
    errorMessage.value = '';
    try {
        await authStore.login({ login: login.value, password: password.value });
        router.push('/');
    } catch (error) {
        if (error.response?.status === 401) {
            errorMessage.value = 'Неверный логин или пароль';
        } else {
            errorMessage.value = error.response?.data?.detail || 'Ошибка при входе';
        }
    } finally {
        loading.value = false;
    }
};
</script>

```

Серверный обработчик принимает данные в формате **OAuth2PasswordRequestForm**, аутентифицирует пользователя и возвращает токены в куках.

```

@router.post("/login")
async def login(
    form_data: OAuth2PasswordRequestForm = Depends(),
    response: Response = None,
    auth_service: AuthService = Depends(get_auth_service)

```

```

):
    user = await auth_service.authenticate(form_data.username,
form_data.password)
    if not user:
        raise HTTPException(status_code=401, detail="Неверный логин или
пароль")
    tokens = await auth_service.create_tokens(user.id)
    set_auth_cookies(response, tokens["access_token"],
tokens["refresh_token"])
    return {"message": "Успешный вход", "user": user}

```

3. Проверка текущей сессии и автоматическое обновление токенов

При загрузке приложения фронтенд вызывает эндпоинт `/auth/me` для получения данных о текущем пользователе. Серверный метод `get_me` извлекает токены из кук и при необходимости автоматически обновляет их.

```

@router.get("/me", response_model=UserResponse)
async def get_me(
    request: Request,
    response: Response,
    auth_service: AuthService = Depends(get_auth_service)
):
    access_token = request.cookies.get("access_token")
    refresh_token = request.cookies.get("refresh_token")

    # Пробуем access_token
    user = await auth_service.get_user_from_token(access_token) if
access_token else None
    if user:
        return user

    # Access не подошёл - пробуем обновить по refresh
    if not refresh_token:
        raise HTTPException(status_code=401, detail="No valid tokens")

    try:
        new_tokens = await auth_service.refresh_token(refresh_token)
        set_auth_cookies(response, new_tokens["access_token"],
new_tokens["refresh_token"])
        user = await
auth_service.get_user_from_token(new_tokens["access_token"])
        if not user:
            raise HTTPException(401, "User not found")
        return user
    except Exception:
        clear_auth_cookies(response)
        raise HTTPException(status_code=401, detail="Session expired, please
login again")

```

4. Выход из системы

При нажатии кнопки выхода вызывается эндпоинт `/auth/logout`, который очищает куки на стороне клиента.

```

@router.post("/logout")

```

```
async def logout(response: Response):
    clear_auth_cookies(response)
    return {"message": "Вы вышли из системы"}
```

Функция `clear_auth_cookies` удаляет обе куки:

```
def clear_auth_cookies(response: Response):
    response.delete_cookie("access_token")
    response.delete_cookie("refresh_token")
```

Заключение по модулю

Представленная реализация обеспечивает безопасную работу с сессией благодаря `httpOnly` кукам, автоматическому обновлению токенов и централизованной обработке аутентификации в сервисе `AuthService`. Фронтенд использует хранилище `Pinia` для управления состоянием авторизации и единообразной отправки запросов, что упрощает поддержку и масштабирование кода.

4.3 Разработка алгоритма рекомендательной системы

Рекомендательная система BeatMarket реализована без применения машинного обучения и основана на анализе истории взаимодействий пользователя с платформой. Алгоритм персональных рекомендаций находится в классе `RecommendationService` (файл `recommendation.py`).

4.3.1 Алгоритм персональных рекомендаций

Метод `get_personal_recommendation` выполняет следующие шаги:

1. Сбор предпочтений пользователя:

- Из таблицы `purchases` извлекаются все купленные треки, из таблицы `interactions` – все треки, которые пользователь прослушал (тип `listen`).
- Для каждого трека определяется его жанр (`genre_id`) и автор (`author_id`).

2. Подсчёт популярности жанров и авторов:

- Строятся два подзапроса (`UNION`), подсчитывающие количество взаимодействий с каждым жанром и автором.
- Выбираются топ-5 жанров и топ-5 авторов по суммарному количеству взаимодействий.

```
genre_query = (
    select(Track.genre_id, func.count(Track.id).label('cnt'))
    .join(Purchase, Purchase.track_id == Track.id)
    .where(Purchase.user_id == user_id)
    .group_by(Track.genre_id)
    .union_all(
        select(Track.genre_id, func.count(Track.id))
        .join(Interaction, Interaction.track_id == Track.id)
        .where(Interaction.user_id == user_id, Interaction.interaction_type ==
InteractionType.listen)
```

```
        .group_by(Track.genre_id)
    )
).subquery()
```

1. Формирование списка исключений:

- Собираются ID всех треков, которые пользователь уже купил или прослушал, а также ID, переданные в параметре `exclude_ids` (для избежания повторов в рамках одной сессии).

2. Поиск кандидатов:

- Выполняется запрос к таблице `Track` с условием: `genre_id IN (топ_жанры) OR author_id IN (топ_авторы)`.
- Исключаются треки из списка исключений.
- Результат сортируется случайным образом (`ORDER BY random()`) и ограничивается лимитом.

3. Запасной вариант:

- Если ни одного трека не найдено, запрос повторяется без учёта списка исключений (но всё равно исключая уже купленные/прослушанные).
- Если и после этого результат пуст – вызывается метод `get_popular_tracks`, возвращающий просто популярные треки.

4. Загрузка связанных данных:

- Полученные ID треков используются для финального запроса с подгрузкой жанра и автора через `selectinload`.

```
query = select(Track).where(Track.id.in_(track_ids)).options(
    selectinload(Track.genre),
    selectinload(Track.author)
)
```

4.3.2 Алгоритм популярных треков

Для неавторизованных пользователей или при отсутствии истории используется метод `get_popular_tracks`:

- Из таблицы `Track` выбираются 20 треков с наибольшим количеством прослушиваний (`plays`).
- Из этой выборки случайным образом берутся `limit` треков (исключая переданные `exclude_ids`).

4.3.3 Эндпоинты рекомендаций

Роутер `recommendations.py` предоставляет два основных эндпоинта:

- `GET /recommendations/personal` – персональные рекомендации. Принимает параметры `limit` и `exclude` (список ID через запятую). Если пользователь не

авторизован, автоматически переключается на популярные треки.

- `GET /recommendations/popular` – популярные треки.

Оба эндпоинта возвращают массив объектов `TrackResponse`, содержащих всю необходимую информацию для отображения карточки трека.

4.3.4 Интеграция с фронтом

На клиентской стороне модальное окно рекомендаций (компонент `RecommendationModal.vue`) при нажатии кнопки «Получить рекомендацию» отправляет запрос к `/recommendations/personal?limit=1`. При повторных нажатиях («Другой трек») к запросу добавляется параметр `exclude`, в который попадает ID уже показанного трека. Это гарантирует, что пользователь не увидит один и тот же трек дважды подряд в рамках одной сессии.

Таким образом, рекомендательная система BeatMarket эффективно использует накопленные данные о поведении пользователей для формирования персонализированных предложений, не требуя при этом ресурсоёмких ML-моделей. Алгоритм легко масштабируется и может быть доработан путём добавления весовых коэффициентов для разных типов взаимодействий (например, покупка важнее прослушивания).

4.4 Разработка серверной части

Серверная часть информационной системы BeatMarket представляет собой асинхронное веб-приложение, построенное на фреймворке **FastAPI**. Выбор FastAPI обусловлен его высокой производительностью (сравнимой с Node.js и Go), автоматической генерацией OpenAPI-спецификации, встроенной валидацией данных через Pydantic и превосходной поддержкой асинхронности. Для взаимодействия с базой данных используется **SQLAlchemy** в асинхронном режиме с драйвером `asyncpg` для PostgreSQL, что позволяет не блокировать цикл событий при выполнении запросов. Аутентификация реализована на основе JWT-токенов с хранением в `httpOnly cookies`, а хеширование паролей выполняется через `passlib` с алгоритмом `bcrypt`.

Сервер предоставляет следующие возможности:

- Регистрация нового пользователя с проверкой уникальности логина и email и автоматической установкой токенов в куки.
- Вход в систему с выдачей пары токенов (access + refresh).
- Автоматическое обновление access-токена по refresh-токену из куки.
- Просмотр каталога треков с поиском и фильтрацией по жанрам, BPM, длительности.
- Потокосное воспроизведение MP3-файлов с подсчётом прослушиваний.
- Добавление треков в избранное и подписка на авторов.
- Покупка трека с выбором типа лицензии и автоматической генерацией договора.

- Административная панель для управления пользователями, жанрами и треками.
- Персонализированная рекомендательная система, анализирующая историю прослушиваний и покупок.

Все эндпоинты возвращают данные в формате JSON, а защищённые маршруты требуют наличия валидного access-токена в httpOnly куке.

4.4.1 Разработка базы данных

Файл `models.py` содержит определения всех таблиц базы данных в виде SQLAlchemy-моделей с использованием декларативного стиля (наследование от `Base`). Для перечислимых типов используются Python-перечисления (`enum.Enum`), которые мапятся в нативные типы PostgreSQL через `sqlalchemy.Enum`.

Основные сущности предметной области:

- **User** — пользователь системы. Хранит учётные данные, роль (user/author/admin), флаг активности и ссылку на аватар.
- **Author** — профиль автора, связанный один-к-одному с User через `user_id`.
- **Genre** — жанр музыкального произведения.
- **Track** — музыкальный трек. Содержит название, цену, длительность, BPM, пути к файлам обложки и MP3, а также внешние ключи на жанр и автора.
- **Favorite** — связь «пользователь — трек» для избранного.
- **Purchase** — запись о покупке трека с указанием типа лицензии, суммы и статуса.
- **Contract** — договор, привязанный к покупке (один-к-одному), с уникальным номером и путём к PDF-файлу.
- **Interaction** — запись о взаимодействии пользователя с треком (прослушивание, добавление в избранное, покупка). Используется для сбора статистики и рекомендаций.
- **Subscription** — подписка пользователя на автора.

Все связи между таблицами описаны через `relationship`, что позволяет легко загружать связанные объекты с помощью `selectinload` или `joinedload`.

Пример определения модели `Track` с перечислениями и индексами:

```
class Track(Base):
    __tablename__ = 'tracks'

    id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
    title: Mapped[str] = mapped_column(String(200), nullable=False)
    cover_url: Mapped[str | None] = mapped_column(String(200))
    duration_seconds: Mapped[int | None] = mapped_column(Integer)
    created_date: Mapped[DateTime | None] = mapped_column(DateTime)
    added_date: Mapped[DateTime] = mapped_column(DateTime(timezone=True),
server_default=func.now())
    mp3_file_url: Mapped[str] = mapped_column(String(200), nullable=False)
```



```

price: Mapped[float] = mapped_column(Float, nullable=False, default=0.0)
plays: Mapped[int] = mapped_column(Integer, nullable=False, default=0.0)
bpm: Mapped[int] = mapped_column(Integer, nullable=True, default=0.0)

genre_id: Mapped[int] = mapped_column(Integer, ForeignKey('genres.id'),
nullable=False)
author_id: Mapped[int] = mapped_column(Integer, ForeignKey('authors.id'),
nullable=False)

genre = relationship("Genre", back_populates="tracks")
author = relationship("Author", back_populates="tracks")
favorites = relationship("Favorite", back_populates="track", cascade="all,
delete-orphan")
purchases = relationship("Purchase", back_populates="track", cascade="all,
delete-orphan")
interactions = relationship("Interaction", back_populates="track",
cascade="all, delete-orphan")

__table_args__ = (
    Index('idx_track_title', 'title'),)

```

Использование `Mapped` и аннотаций типов соответствует стилю SQLAlchemy 2.0 и обеспечивает строгую типизацию.

4.4.2 Сервис аутентификации

`AuthService` инкапсулирует всю логику, связанную с регистрацией, аутентификацией и управлением JWT-токенами. Он принимает сессию БД в конструктор и предоставляет следующие методы:

- **register(user_data: UserCreate)** — создаёт нового пользователя, предварительно проверяя уникальность логина и email, хэширует пароль с помощью `passlib` и сохраняет запись в БД.
- **authenticate(login: str, password: str)** — находит пользователя по логину (или email) и проверяет пароль через `verify_password`.
- **create_tokens(user_id: int)** — генерирует пару токенов: access (срок действия 1 час) и refresh (7 дней). В payload access-токена включается `user_id`.
- **refresh_token(refresh_token: str)** — проверяет валидность refresh-токена и выпускает новую пару.
- **get_user_from_token(token: str)** — извлекает `user_id` из payload и возвращает объект пользователя.

Роутер `auth.py` определяет эндпоинты:

- **POST /auth/register** — регистрация. После успешного создания пользователя сразу вызывается `create_tokens`, и токены устанавливаются в httpOnly куки через функцию `set_auth_cookies`.

```

@router.post("/register", response_model=UserResponse,
status_code=status.HTTP_201_CREATED)
async def register(
    user_data: UserCreate,
    response: Response,

```

```

    auth_service: AuthService = Depends(get_auth_service)
):
    user = await auth_service.register(user_data)
    tokens = await auth_service.create_tokens(user.id)
    set_auth_cookies(response, tokens["access_token"],
tokens["refresh_token"])
    return user

```

- **POST /auth/login** — вход. Принимает данные в формате `OAuth2PasswordRequestForm` (для совместимости со Swagger UI), аутентифицирует пользователя и устанавливает куки.
- **GET /auth/me** — возвращает данные текущего пользователя. Если access-токен истёк, но присутствует refresh-токен, эндпоинт автоматически обновляет пару и возвращает пользователя. Это избавляет фронтенд от необходимости самостоятельно отслеживать истечение токенов.

```

@router.get("/me", response_model=UserResponse)
async def get_me(request: Request, response: Response, auth_service:
AuthService = Depends(get_auth_service)):
    access_token = request.cookies.get("access_token")
    refresh_token = request.cookies.get("refresh_token")
    user = await auth_service.get_user_from_token(access_token) if
access_token else None
    if user:
        return user
    if not refresh_token:
        raise HTTPException(status_code=401, detail="No valid tokens")
    new_tokens = await auth_service.refresh_token(refresh_token)
    set_auth_cookies(response, new_tokens["access_token"],
new_tokens["refresh_token"])
    user = await auth_service.get_user_from_token(new_tokens["access_token"])
    return user

```

- **POST /auth/logout** — удаляет куки, завершая сессию.

Функции `set_auth_cookies` и `clear_auth_cookies` управляют `httpOnly` куками с флагами `httpOnly=True`, `secure` (в production) и `samesite='lax'`.

4.4.3 Сервис работы с треками

`TrackService` отвечает за управление музыкальным контентом. В конструктор передаётся репозиторий `TrackRepository` и `FileService` для сохранения файлов.

Метод `create_track` выполняет загрузку нового трека автором:

- Проверяет, что пользователь имеет роль `author` и связанный профиль `Author`.
- Сохраняет загруженные файлы обложки и МРЗ в папки `storage/covers/` и `storage/tracks/` через `FileService`.
- Создаёт запись в таблице `Track`, связывая её с жанром и автором.
- При необходимости вычисляет длительность МРЗ-файла (с помощью библиотеки `mutagen`).

Метод `search_tracks` реализует гибкий поиск с фильтрацией:

- По текстовому запросу (`query`) ищет совпадения в названии трека.
- Фильтрует по списку ID жанров.
- Поддерживает диапазоны BPM и длительности.
- Сортирует по популярности (поле `plays`), новизне (`added_date`), цене (возрастание/убывание).

Роутер `tracks.py` предоставляет эндпоинты:

- `GET /tracks/` — список всех треков с пагинацией.
- `GET /tracks/search` — поиск с фильтрами.
- `GET /tracks/popular` и `/tracks/new` — популярные и новые треки.
- `POST /tracks/` — создание трека (только для авторов). Принимает `multipart/form-data` с полями `title`, `genre_id`, `price`, `bpm` и файлами `cover`, `mp3`.
- `GET /tracks/{track_id}/stream` — потоковое воспроизведение МРЗ. Возвращает `FileResponse` с MIME-типом `audio/mpeg`, что позволяет браузеру проигрывать трек через встроенный аудиоплеер.

```
@router.get("/{track_id}/stream")
async def stream_track(track_id: int, service: TrackService =
    Depends(get_track_service)):
    track = await service.get_track(track_id)
    if not track:
        raise HTTPException(status_code=404, detail="Track not found")
    file_path = track.mp3_file_url
    if not os.path.exists(file_path):
        raise HTTPException(status_code=404, detail="Audio file not found")
    return FileResponse(file_path, media_type="audio/mpeg",
        filename=f"{track.title}.mp3")
```

- `GET /tracks/{track_id}/download` — скачивание трека (только для авторизованных пользователей, купивших трек или являющихся его автором).

4.4.4 Сервис покупок и генерация договоров

Процесс покупки реализован в `PurchaseService` с привлечением `ContractService` для создания договора.

Метод `purchase_track_with_data` обрабатывает запрос на покупку:

- Проверяет, не куплен ли уже этот трек данным пользователем (для авторизованных).
- Создаёт запись в `Purchase` с указанием суммы, типа лицензии (`standard` / `extended` / `exclusive`), статуса `completed`.
- Генерирует уникальный номер договора в формате `CONTRACT-{user_id}-{track_id}-{timestamp}`.
- Вызывает `ContractService.create_contract`, который создаёт PDF-документ с текстом договора (используя шаблон) и сохраняет его в папку

`uploads/contracts/`. Путь к файлу записывается в поле `document_url` таблицы `Contract`.

Эндпоинт `POST /purchase/purchases` принимает JSON с `track_id`, `license_type`, а также опциональные `buyer_name` и `buyer_email` (для гостевой покупки). После успешного создания покупки возвращается информация о договоре и ссылка для скачивания.

Скачивание договора реализовано через `GET /purchase/{purchase_id}/contract`.

Эндпоинт проверяет, что текущий пользователь является владельцем покупки, и отдаёт PDF-файл через `FileResponse`.

4.4.5 Рекомендательная система

`RecommendationService` реализует алгоритм персональных рекомендаций без использования машинного обучения, основанный на анализе предпочтений пользователя.

Алгоритм `get_personal_recommendation`:

1. Собирает все треки, которые пользователь купил или прослушал (из таблиц `Purchase` и `Interaction`).
2. Вычисляет топ-5 жанров и топ-5 авторов на основе частоты встречаемости.
3. Формирует список ID треков, которые уже взаимодействовал пользователь (для исключения).
4. Выполняет запрос к таблице `Track`: `genre_id IN (топ_жанры) OR author_id IN (топ_авторы) AND id NOT IN (исключённые)`.
5. Сортирует результат случайным образом и ограничивает лимитом.
6. Если результат пуст, снимает фильтр по жанрам/авторам (оставляя только исключение уже купленных/прослушанных).

Для неавторизованных пользователей или при отсутствии истории вызывается `get_popular_tracks`, который возвращает треки с наибольшим количеством прослушиваний (`plays`).

Эндпоинты находятся в `recommendations.py`:

- `GET /recommendations/personal?limit=1&exclude=...` — возвращает один персонально рекомендованный трек. Параметр `exclude` принимает список ID уже показанных треков, чтобы избежать повторов в рамках одной сессии.
- `GET /recommendations/popular` — популярные треки.
- `GET /recommendations/stream/{track_id}` — потоковое воспроизведение рекомендованного трека (аналогично `/tracks/{track_id}/stream`, но вынесено в отдельный роутер для удобства).

4.4.6 Личный кабинет пользователя и автора

Роутеры `user.py` и `author.py` предоставляют эндпоинты для управления профилем и

получения персональных данных.

Для обычного пользователя (/users):

- GET /users/me — профиль текущего пользователя.
- PUT /users/me — обновление имени, email и аватара.
- GET /users/me/purchases — список купленных треков.
- GET /users/me/favorites — список избранного.
- GET /users/me/subscriptions — список подписок на авторов.
- POST /users/me/subscriptions/{author_id} — подписка на автора.

Для автора (/authors):

- GET /authors/me — расширенный профиль автора с дополнительными полями: количество подписчиков, общий заработок, количество треков, средний рейтинг.
- GET /authors/me/tracks — список треков автора.
- GET /authors/me/stats — подробная статистика по продажам и прослушиваниям.

Все эти эндпоинты защищены зависимостью `get_current_user`, которая извлекает пользователя из access-токена в куке.

4.4.7 Административная панель

Для администраторов реализован отдельный набор эндпоинтов (в файле `admin.py`, не предоставлен в фрагментах, но упоминается в структуре). Типичные возможности:

- Управление пользователями: блокировка/разблокировка, изменение роли.
- Управление жанрами: CRUD.
- Управление треками: редактирование, удаление.
- Просмотр агрегированной статистики: выручка, средний чек, топ покупателей, топ слушателей.

Статистика собирается с помощью `StatsService`, который выполняет агрегирующие запросы к БД (например, сумма `amount` из `Purchase`, количество уникальных `user_id` за период и т.д.).

4.5 Разработка клиентской части

Клиентская часть информационной системы BeatMarket реализована в виде одностраничного приложения (SPA) на фреймворке **Vue 3** с использованием **Composition API** и языка **TypeScript** (в проекте частично применяется JavaScript, но структура поддерживает строгую типизацию). Для управления состоянием приложения задействована библиотека **Pinia**, для маршрутизации – **Vue Router**, а для выполнения HTTP-запросов – **axios**.

Архитектура фронтенда построена по модульному принципу:

- **Страницы (views)** – компоненты, соответствующие отдельным маршрутам (`Home.vue`, `Search.vue`, `Profile.vue`).
- **Переиспользуемые компоненты** – `TrackCard.vue`, `ArtistDashboard.vue` и др.
- **Composables** – функции, инкапсулирующие повторно используемую логику (например, `useSearch`, `useProfile`).
- **Хранилища (stores)** – `authStore` для управления аутентификацией, `playerStore` для плеера.
- **API-клиент** – настроенный экземпляр `axios` с базовым URL и перехватчиками для обработки ошибок.

4.5.1 Главная страница и каталог

Компонент `Home.vue` формирует лендинг платформы. Он состоит из трёх секций:

1. **Hero-секция** с призывом к действию и анимированным градиентом.
2. **Популярные треки** – загружаются с эндпоинта `/tracks/popular`.
3. **Подборки по жанрам** – жанры получаются с `/genres`, для каждого подставляется тематическая обложка из предопределённого маппинга.
4. **Новые релизы** – запрос к `/tracks/new`.

```
// Home.vue (фрагмент)
const fetchPopular = async () => {
  const { data } = await api.get('/tracks/popular?limit=4');
  popularTracks.value = data;
};
```

Для отображения карточек треков используется компонент `TrackCard`, который принимает объект трека и генерирует единообразную карточку с обложкой, названием, ценой и кнопками взаимодействия.

Поиск и фильтрация реализованы в компоненте `Search.vue`. Он предоставляет:

- Строку поиска с живым обновлением (debounce не показан, но может быть добавлен через `watch` с задержкой).
- Боковую панель фильтров (жанры, диапазоны BPM и длительности).
- Сортировку результатов по популярности, новизне, цене.
- Переключение между сеткой и списком.
- Чипсы активных фильтров с возможностью быстрого сброса.

При изменении любого параметра фильтрации вызывается функция `fetchTracks`, которая отправляет GET-запрос на `/tracks/search` с соответствующими query-параметрами.

```
// Пример формирования запроса из Search.vue (через useSearch)
```

```
const params = {
  query: searchQuery.value,
  genre_ids: selectedGenres.value.map(g => g.id),
  bpm_min: bpmRange.value[0],
  bpm_max: bpmRange.value[1],
  duration_min: durationRange.value[0],
  duration_max: durationRange.value[1],
  sort_by: sortBy.value,
  skip: skip.value,
  limit: limit.value
};
```

4.5.2 Личный кабинет и профиль автора

Компонент `Profile.vue` представляет собой многофункциональный личный кабинет, адаптирующийся под роль пользователя. Он содержит:

- **Hero-секцию** с аватаром, именем, ролью и статистикой (количество треков, подписчиков, покупок).
- **Табы** для переключения между разделами: «Мои данные», «Покупки», «Избранное», «Подписки», а для авторов дополнительно – «Студия продюсера», «Мои треки», «Загрузить бит».
- **Модальное окно** редактирования профиля с загрузкой нового аватара.
- **Форму загрузки трека** с предпросмотром обложки.

Все данные подгружаются через кастомный composable `useProfile`, который агрегирует запросы к эндпоинтам `/users/me`, `/users/me/purchases`, `/users/me/favorites` и т.д.

Пример загрузки треков автора:

```
const fetchAuthorTracks = async () => {
  const response = await api.get("/tracks/me");
  authorTracks.value = response.data;
};
```

Загрузка нового трека выполняется через POST-запрос с `multipart/form-data`:

```
const formData = new FormData();
formData.append("title", newTrack.value.title);
formData.append("genre_id", newTrack.value.genre_id);
formData.append("price", newTrack.value.price);
formData.append("cover", newTrack.value.cover_file);
formData.append("mp3", newTrack.value.mp3_file);
await api.post("/tracks", formData);
```

После успешной загрузки пользователь перенаправляется на вкладку «Мои треки».

4.5.3 Взаимодействие с сервером и аутентификация

Все запросы к API проходят через экземпляр `axios`, в который добавлен перехватчик для автоматической подстановки CSRF-токена (при необходимости) и обработки

ошибок 401. Токены доступа хранятся исключительно в `httpOnly cookies`, поэтому фронтенд не управляет ими напрямую – достаточно просто отправлять запросы с `withCredentials: true`.

Пример конфигурации `axios`:

```
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  withCredentials: true,
});
```

При получении ошибки 401 (например, из-за истечения `access-токена`) срабатывает механизм обновления через эндпоинт `/auth/refresh`, который автоматически устанавливает новые куки. Это реализовано в серверной части, но фронтенд может повторить исходный запрос после обновления токена.

Заключение

Производственная практика сыграла ключевую роль в моём профессиональном развитии, предоставив возможность не только применить теоретическую базу на практике, но и освоить реальные методы работы в сфере проектирования информационных систем и написания программного кода. В рамках практического обучения были успешно решены задачи по построению системной архитектуры, созданию функциональных модулей и настройке взаимодействия компонентов.

Существенное внимание уделялось выработке профессиональных умений, соответствующих образовательным стандартам. Важным результатом стало изучение полного цикла создания клиент-серверного приложения — начиная со сбора требований и заканчивая построением элементарных составных частей. В ходе деятельности использовались актуальные инструменты web-разработки, средства администрирования баз данных и системы отслеживания версий, что позволило сформировать востребованные на рынке труда компетенции.

Обучающийся _____ / _____ /
подпись расшифровка подписи